

Scalability & Future Plan

Date	7 July 2026
Team ID	
Project Name	Personal Network Assistant
Maximum Marks	1 Mark

Scalability & Future Plan:

This document outlines how the current project solution can be scaled to handle larger user bases, increased data loads, or extended features in the future. It also captures the team's roadmap for enhancing and evolving the project beyond its current state.

Current System Limitations:

S.No	Limitation	Impact	Priority to Address (High / Medium / Low)
1	Local or single-stack deployment	Suitable for development, but lacks load balancing, multi-instance failover, and production traffic management.	High
2	SQLite-based local storage	Heavy concurrent writes or multiple backend instances may cause locking and durability limitations.	High
3	Limited observability and synchronous external-AI operations	Slow model calls, production issues, and capacity trends may be harder to detect and diagnose.	Medium

Scalability Plan:

S.No	Scalability Aspect	Current State	Proposed Upgrade / Solution
1	User Load	FastAPI backend runs locally or through one Docker Compose stack with a separate Streamlit frontend.	Use a production ASGI server, reverse proxy, load balancer, horizontally scaled stateless API instances, and shared external storage.
2	Data Storage	SQLAlchemy uses SQLite by default, with the database stored locally or in a Docker volume.	Migrate to PostgreSQL, add migrations, connection pooling, indexes, backups, and read replicas only when needed.
3	Performance	Requests are handled synchronously, and external-AI operations may have variable response times.	Add caching, background queues, asynchronous I/O, timeouts, retries, query optimization, pagination, and endpoint-specific load tests.
4	Security	Bearer-token authentication, bcrypt hashing, protected routes, validation, selected rate limiting, and CI security checks are present.	Add HTTPS, managed secrets, token refresh or rotation, stricter CORS, audit logging, RBAC if needed, dependency scanning, and centralized security monitoring.

Future Roadmap:

Phase	Planned Feature / Enhancement	Target Timeline	Expected Impact
Phase 1	PostgreSQL migration, environment-based configuration, database migrations, HTTPS deployment, and automated backup support.	1-2 months	Improves reliability, data durability, deployment consistency, and readiness for multiple users.

Phase 2	Caching, asynchronous processing, request timeouts, database indexing, pagination, and improved AI-operation handling.	2-3 months	Reduces response latency, improves concurrency, and prevents long-running requests from blocking users.
Phase 3	Centralized logging, metrics dashboards, tracing, health monitoring, alerting, and expanded load testing.	3-4 months	Improves production monitoring, diagnosis, capacity planning, and incident response.
Phase 4	Richer personalization, recommendation improvements, configurable preferences, team support, and privacy-aware analytics.	4-6 months	Improves user value, personalization quality, engagement, and suitability for broader professional use.